Proyecto 1 · DFSha

Diseño e implementación de un sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad

1. Objetivo general

Diseñar e implementar un sistema de archivos distribuido (DFS) que permita almacenar y acceder a archivos grandes mediante múltiples nodos, con particionamiento, replicación, disponibilidad, rendimiento, transparencia y seguridad. Tanto la escritura como la lectura deben involucrar varios nodos, según el diseño adoptado.

2. Organización y reglas de trabajo

El proyecto consta de dos fases evaluables: (I) diseño arquitectónico individual (40 %) y (II) implementación y validación grupal (60 %). Cada estudiante presenta una propuesta autónoma antes de que su equipo defina la solución común. La opción arquitectónica asignada por el profesor (opción 1 u opción 2) debe respetarse en ambas fases.

- Fase I: no requiere código funcional. Se evalúan decisiones, modelos, protocolos, algoritmos y justificación técnica individual.
- Fase II: requiere código ejecutable, pruebas reproducibles, evidencias y defensa de las decisiones del equipo.
- El diseño grupal puede adoptar o combinar propuestas individuales; debe registrar qué decisiones cambian y por qué.
- Todos los integrantes deben poder explicar la arquitectura completa y demostrar su contribución.

3. Requisitos comunes a las dos fases

3.1 Requisitos funcionales

- RF1. Gestión del sistema de archivos jerárquico: ls, cd, mkdir, rmdir, rm y operaciones equivalentes. Cada usuario autenticado tendrá un espacio lógico de almacenamiento propio y, por defecto, únicamente podrá acceder y modificar sus archivos. No se exige implementar mecanismos para compartir archivos entre diferentes usuarios.
- RF2. Transferencia de archivos completos: send() y receive()/recv() o comandos equivalentes put/get.
- RF3. Acceso a archivos: open(), close(), read(), write(), lock() y otras primitivas justificadas. El equipo debe declarar explícitamente si write() admite creación, reemplazo, anexado y/o modificación parcial por bloque; no se presupone semántica POSIX completa.

3.2 Requisitos no funcionales

- RNF1. Escalabilidad en usuarios, número y tamaño de archivos.
- RNF2. Alta disponibilidad y redundancia ante fallos de nodos.
- RNF3. Consistencia de datos y tratamiento de réplicas.
- RNF4. Particionamiento: distintos archivos y los bloques de un mismo archivo deben distribuirse entre varios nodos.

- RNF5. Rendimiento y acceso concurrente a archivos y datos dentro de un archivo.
- RNF6. Seguridad: cifrado en tránsito y reposo, autenticación, autorización y protección de comunicaciones entre nodos.
- RNF7. Transparencia de acceso y localización, sin direcciones de nodos codificadas de forma estática en el cliente.

3.3 Arquitecturas asignables

- Opción 1 — Cliente/servidor con servicio distribuido entre servidores (S2S), operado dentro de una red privada real o virtual.
- Opción 2 — P2P con superpares de organizaciones diferentes, sobre una red superpuesta e interconexión privada virtual.

4. Fase I — Diseño individual (40 %)

Cada estudiante elaborará de manera independiente una propuesta técnica completa para el DFSha, sin necesidad de programar. Debe explicitar supuestos, limitaciones y decisiones, y vincularlas con los requisitos anteriores.

4.1 Actividades individuales

- Definir el servicio y sus casos de uso: actores, operaciones RF1–RF3, secuencias de carga, lectura, escritura y borrado.
- Proponer la arquitectura asignada: nodos, responsabilidades, metadatos, plano de control y plano de datos; presentar diagrama de componentes y despliegue.
- Diseñar la distribución de archivos en bloques, localización de bloques, metadatos, réplicas y reconstrucción del archivo.
- Especificar protocolos y APIs entre cliente, controladores y nodos de datos (REST, gRPC, TCP o MOM, según corresponda), con ejemplos de mensajes y respuestas de error.
- Definir semántica de read()/write(): granularidad, operaciones admitidas, visibilidad de escrituras y comportamiento de accesos concurrentes.
- Diseñar mecanismos de consistencia, exclusión mutua o control de versiones, integridad y recuperación de escrituras interrumpidas.
- Describir tolerancia a fallos, detección de nodos caídos, recuperación y mantenimiento de disponibilidad.
- Diseñar autenticación, permisos, cifrado en tránsito y reposo, y protección de comunicaciones internas.
- Definir métricas y experimentos previstos: latencia, throughput, efecto de la concurrencia, fallos, replicación y escalabilidad; justificar compromisos técnicos.

4.2 Escenarios obligatorios de razonamiento

- Dos procesos intentan escribir el mismo bloque de un archivo simultáneamente: ¿qué sucede y por qué?
- Una réplica está desconectada durante una escritura: ¿qué observan los lectores y cómo se recupera?
- Un nodo falla después de recibir parte de un bloque: ¿cómo se evita publicar un archivo incompleto?
- El cliente desconoce la ubicación física de los bloques: ¿cómo los descubre dinámicamente?

4.3 Entregable individual

Documento técnico individual (PDF o Word), sugerido de 6–10 páginas sin anexos, con diagramas legibles, especificación de APIs, decisiones argumentadas, análisis de escenarios y referencias. No se exige implementación en esta fase.

5. Fase II — Implementación grupal (60 %)

Cada equipo compara las propuestas individuales, selecciona y documenta una arquitectura final y construye un prototipo distribuido verificable. No basta una simulación local monolítica como producto final.

5.1 Actividades grupales

- Elaborar matriz comparativa breve de las propuestas individuales y un registro de decisiones arquitectónicas del equipo.
- Implementar CLI o API de cliente para RF1 y RF2 y las operaciones de RF3 acordadas, con la semántica declarada.
- Implementar nodos de control y/o superpares y nodos de datos, según la arquitectura asignada; exponer APIs y desplegar procesos o contenedores separados.
- Implementar particionamiento de cada archivo en varios nodos, almacenamiento de metadatos, distribución y recuperación de bloques.
- Implementar replicación, política de consistencia, manejo de concurrencia y recuperación de fallos, con evidencia observable.
- Implementar mecanismos básicos de autenticación, autorización y cifrado según el diseño y las posibilidades del entorno.
- Desplegar en máquinas virtuales de AWS Academy, nativamente o con Docker; demostrar comunicación real entre nodos por red.
- Medir y analizar desempeño, disponibilidad y corrección con pruebas repetibles, incluyendo escenarios de escritura concurrente y caída de nodos.

5.2 Pruebas mínimas exigidas

Prueba	Evidencia esperada
Carga y recuperación	Archivo grande particionado en varios nodos; archivo recuperado idéntico (hash).
Acceso RF3	Demostración de read()/write() con la semántica declarada.
Concurrencia	Dos clientes sobre un mismo archivo/bloque; resultado acorde con la política diseñada.
Replicación y fallos	Caída de un nodo y lectura o recuperación según las garantías prometidas.
Consistencia	Comportamiento observable cuando una réplica se retrasa o desconecta.
Seguridad	Prueba de acceso autorizado y denegado; evidencia de protección del canal y almacenamiento.
Rendimiento	Mediciones de latencia/throughput y comparación bajo al menos dos condiciones.

5.3 Entregables grupales

- Repositorio GitHub con código, instrucciones reproducibles de instalación, configuración y ejecución, y contribuciones identificables.

- Informe técnico final: arquitectura consensuada, cambios frente a los diseños individuales, APIs, algoritmos, despliegue, pruebas, resultados y limitaciones.
- Video demostrativo de 10–15 minutos: arquitectura, ejecución distribuida, fallos y resultados medidos.
- Defensa oral o demostración en la que el profesor pueda interrogar individualmente a cualquier integrante.

6. Evaluación

Componente	Criterio	Peso
Individual	Definición del servicio, RF y casos de uso	8 %
Individual	Arquitectura, distribución y protocolos	12 %
Individual	Consistencia, concurrencia y tolerancia a fallos	12 %
Individual	Seguridad, métricas y justificación	8 %
Grupal	Implementación funcional RF1–RF3	15 %
Grupal	Distribución, escalabilidad y comunicaciones	12 %
Grupal	Alta disponibilidad, replicación y consistencia	15 %
Grupal	Seguridad y pruebas de rendimiento	10 %
Grupal	Documentación, video y defensa	8 %
TOTAL		100 %

Nota de evaluación: la nota de diseño individual no se sustituye por el diseño adoptado por el equipo. En la fase grupal se evaluará tanto el producto verificable como la capacidad de explicar las decisiones y la participación demostrable de sus integrantes.

7. Aclaraciones de alcance

- Las garantías de consistencia deben formularse de manera comprobable (por ejemplo, cuándo una escritura se considera confirmada y qué pueden leer otros clientes).
- lock() puede implementarse como una primitiva explícita o justificarse un mecanismo equivalente; debe documentarse su semántica.
- La integridad (hashes, validaciones y escrituras atómicas) no sustituye la coordinación de escrituras concurrentes.
- Las herramientas sugeridas son Linux, Docker, FastAPI/Flask, gRPC y, cuando proceda, RabbitMQ/Kafka; su elección debe justificarse.
- Cualquier simplificación de RF3 o de garantías no funcionales debe quedar declarada y aprobada por el profesor antes de la implementación.